

I understand—word wrap makes code and folder trees look like a mess. I will keep this very clean and wide so it is easy to read on any screen.

Here is the **Industrial Production Structure** for your MERN + Next + Postgres + Python project.

The Root Project Structure

```
/production-project
├── apps/
│   ├── web/                <-- [NEXT.JS FRONTEND]
│   ├── api/                <-- [NODE.JS BACKEND]
│   └── ml-service/         <-- [PYTHON ML MODELS]
├── packages/
│   ├── database/          <-- [SHARED POSTGRES LOGIC/PRISMA]
│   ├── config/            <-- [SHARED ESLINT/TSCONFIG]
│   └── types/              <-- [SHARED TS INTERFACES]
├── docker-compose.yml      <-- [ORCHESTRATION]
├── .env.example            <-- [ENV TEMPLATE]
└── package.json            <-- [MONOREPO ROOT]
```

1. The Frontend (apps/web)

Using Next.js for the UI.

```
/apps/web
├── src/
│   ├── app/                # Next.js App Router (Pages)
│   ├── components/         # UI Components (Buttons, Modals)
│   ├── hooks/              # Custom React Hooks
│   ├── lib/                # API Fetching logic (Axios/Fetch)
│   └── store/               # State Management (Zustand/Redux)
└── next.config.js
```

2. The Backend (apps/api)

Using Node.js/Express for the core business logic.

```
/apps/api
├── src/
│   ├── controllers/        # Handles Requests & Responses
│   ├── routes/             # API Route Definitions
│   ├── services/           # Heavy logic & Database calls
│   ├── middlewares/        # Auth & Validation (JWT, Zod)
│   └── index.ts             # Server Entry Point
└── tsconfig.json
```

3. The ML Service (apps/ml-service)

Using Python/FastAPI for Machine Learning.

```
/apps/ml-service
├─ main.py           # FastAPI entry point
├─ requirements.txt  # Python dependencies
├─ models/          # Trained model files (.pkl)
└─ utils/           # Data preprocessing scripts
```

4. The Shared Database (packages/database)

This is the "Industrial" secret. You define the DB once.

```
/packages/database
├─ schema.prisma      # PostgreSQL Schema
├─ index.ts           # Exported DB Client
└─ migrations/       # SQL Migration history
```

Why this works for you:

1. **MERN + Next:** Your web is Next.js, and your api is the Node/Express part of MERN.
 2. **PostgreSQL:** The packages/database folder ensures both the web and api see the same data structure.
 3. **Gradual Updates:** You can start with web and api. When you are ready for ML, you just start working in the ml-service folder.
 4. **No Confusion:** Each folder has **one** job.
- Does this cleaner, non-wrapped view help you visualize it better? 